Lab4

832204116 张林奕涵

Lab 4

2. $A_{TM} = \{\langle T, w\rangle \mid T \text{ is a TM and } T \text{ accepts } w\}$

Constructing TM R, let R simulate T.

If $\langle T, w\rangle \in A_{TM}$, T accepts w, R accept.

If $\langle T, w\rangle \notin A_{TM}$, T rejects w or loops, R rejects w or loops.

According to the recognizable definition, R is A_{TM} recognizer.

3. Assume that A_{TM} is decidable and H is a decider for A_{TM}.

H halts and accepts if T accepts w. H halts and rejects if T fails to accept w.

$$H(\langle T, w\rangle) = \begin{cases} \text{accept} & M \text{ accept } w \\ \text{reject} & M \text{ does not accept } w \end{cases}$$

Constructing a new TM D with H as a subroutine.

D = "on input $\langle T\rangle$"

Run H in input $\langle T; \langle T\rangle\rangle$

Output the opposite of what H outputs.

So $D\langle T\rangle = \begin{cases} \text{accept}, & T \text{ does not accept } \langle T\rangle \\ \text{reject}, & T \text{ accepts } \langle T\rangle \end{cases}$

Using diagonalization,

$$D(\langle D\rangle) = \begin{cases} \text{accept}, & D \text{ does not accept } \langle D\rangle \\ \text{reject}, & D \text{ accepts } \langle D\rangle \end{cases}$$

Contradiction.

So A_{TM} is undecidable.

4. $\overline{A_{TM}}$ is not turing recognizable.

Because a language is decidable iff it is recognizable and co-Turing-recognizable, A_{TM} is undecidable and recognizable, $\overline{A_{TM}}$ is not recognizable.

5.

1) Nondeterministic TM M decides a language L iff following 2 conditions are met:

① If $w \in L$, there exists at least one computation branch of M on w that enters an accept state and halts.

② If $w \notin L$, then all computation branches of M on w enter a reject state.

2) Nondeterministic TM M decides a language L iff following 2 conditions are met:

① If $w \in L$, there exists at least one computation branch of M on w that enters an accept state in a finit number of steps.

② If $w \notin L$, then all computation branches of M on w enter a reject state or run forever.

6. Church-Turing Thesis.

If a turing machine can not guarantee to halt on all inputs, it can not be considered as an algorithm.

7. $\overline{H_1} = \{w:$ either w is not the encoding of Turing Machine, or it is the encoding "M" of a Turing Machine that does not halt on "M"$\}$

$H_1 = \{w:$ w is encoding "M" of a Turing Machine that halts on "M"$\}$

$H_1 = \{\langle M \rangle \mid M(\langle M \rangle)$ halts$\}$ is a halting problem.

H_1 is recognizable because a recognizer can simulate M on input $\langle M \rangle$.
If M accept, H_1 accept; if M reject, H_1 reject; if M loops, H_1 loops.
H_1 is undecidable. Using diagonalization method to prove it.
$H_1(H_1)$ contradiction.
According to Theorem 4.22, $\overline{H_1}$ is not recursively-enumerable.